

Compiladores e intérpretes

Pautas Etapa 2 – Analizador Sintáctico

Universidad Nacional del Sur

Departamento de Ciencias e Ingeniería de la Computación

2026



Introducción

- El desarrollo de esta **etapa** del proyecto se **corresponde** con los **temas de Análisis Sintáctico** de la materia
- Te recomendamos que **hayas mirado y tengas frescos** todos los temas allí tratados **antes de arrancar** el diseño/desarrollo de esta etapa

Introducción

- En esta **etapa** hay que **diseñar e implementar** un **Analizador Sintáctico** para **MiniJava**
- El Sintáctico se va a valer del Analizador **Léxico desarrollado en la etapa** anterior para obtener los tokens del programa fuente
- Además, de manera similar a la etapa anterior va ser necesario un **Modulo Principal cliente** del Sintáctico/Léxico que:
 - Manejará la **interfaz** con el **usuario** y
 - **Reportará los errores** detectados por el Sintáctico y el Léxico
 - En caso de no haber errores indicará que el análisis fue exitoso

Diseño de Analizador Sintáctico

Diseño del Analizador Léxico

- **Antes que nada:** leer detalladamente el documento de Rglas de Sintaxis de MiniJava en el Moodle de la Materia

Acá van a encontrar la gramática que usamos para especificar las reglas sintácticas de MiniJava

2 Gramática de MiniJava

Cualquier programa MINIJAVA sintácticamente válido debe ser producto de la gramática que se presenta en esta sección. La gramática sigue la notación BNF-extendida, donde:

terminal	es un símbolo terminal
<Class>	es un símbolo no terminal (además la primer letra es una mayúscula)
ϵ	representa la cadena vacía
$X ::= Y$	representa una producción
$X ::= Y Z$	es una abreviación de $X \rightarrow Y$ o $X \rightarrow Z$

Producciones BNF

El no terminal de inicio es <Inicial> y las producciones de la Gramática de MINIJAVA son:

```
<Inicial> ::= <ListaClases>

<ListaClases> ::= <Clase> <ListaClases> | <Clase>

<Clase> ::= class idClase <Herencia> { <ListaMiembros> }

<Herencia> ::= extends idClase |  $\epsilon$ 

<ListaMiembros> ::= <Miembro> <ListaMiembros> |  $\epsilon$ 
```

Diseño del Analizador Sintáctico

- En base a la **gramática** vamos a diseñar un **analizador sintáctico descendente recursivo**
- Pero para poder hacer eso tenemos que **asegurarnos** que la **gramática** sea **LL(1)**...

y adivinen que...

Lamentablemente la gramática de
MiniJava **NO** es **LL(1)**

Diseño del Analizador Sintáctico

- Lo que hay que hacer es **modificar la gramática de MiniJava** para que sea **LL(1)** pero que siga **generando el mismo lenguaje**
- Para esto **primero** tenemos que **identificar** que **producciones** hacen que **no sea LL(1)**:
 - Producciones con **Rekursión a Izquierda**
 - Producciones de un NT donde los **Primeros** de sus partes derechas **comparten tokens** (no están factorizadas)

Diseño del Analizador Sintáctico

- Identificadas las reglas problemáticas, tenemos que aplicar las **técnicas vistas en los tema 3.2.3 y 3.2.4** que modifican la gramática solucionando estos problemas
 - Para el problema de:
 - Producciones con Recursión a Izquierda
 - Producciones de un NT donde los **Primeros** de sus partes derechas **comparten tokens**

Técnica de **Eliminación de Recursión a Izquierda**

Técnicas de **Factorización Simple y Profunda**

Diseño del Analizador Sintáctico

- Identificadas las reglas problemáticas, tenemos que aplicar las **técnicas vistas en los tema 3.2.3 y 3.2.4** que modifican la gramática solucionando estos problemas

- Para el problema de:

- Producciones con Recursión a Izquierda
- Producciones de un NT donde los **Primeros** de sus partes derechas **comparten tokens**

1

Técnica de **Eliminación de Recursión a Izquierda**

2

Técnicas de **Factorización Simple y Profunda**

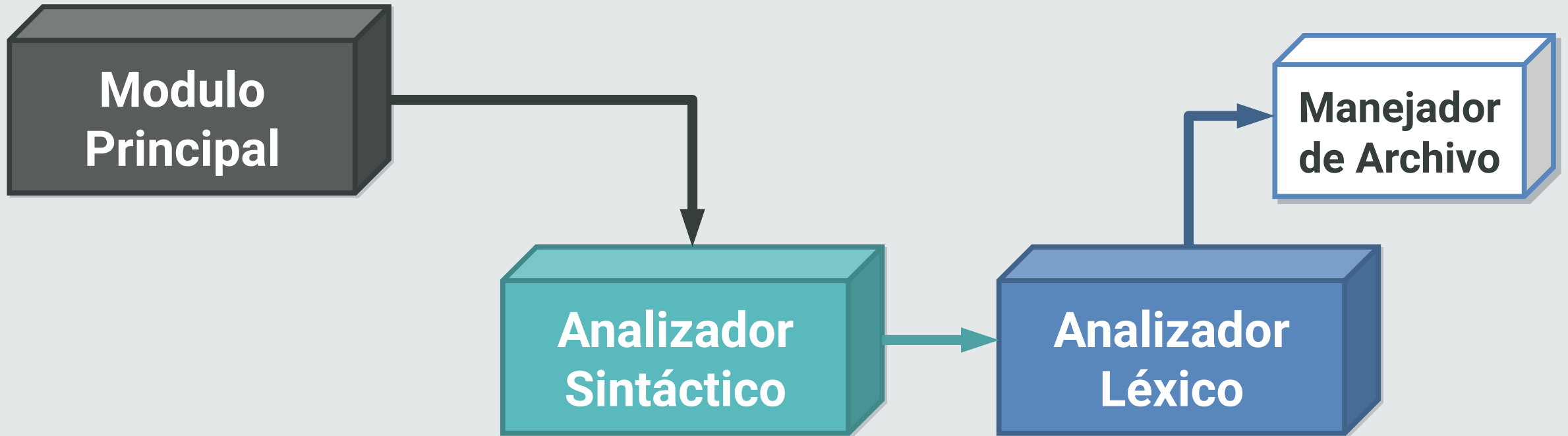
Es importante resolver los problemas en orden!!!

Diseño del Analizador Sintáctico

- Una vez **resueltos los problemas** ya con una gramática LL(1) equivalente a la de MiniJava podemos **diseñar el Sintactico**
- Debe ser un **Analizador Sintáctico Descendente Recursivo**
- Para su diseño pueden utilizar una de las dos alternativas:
 - El **esquema simple** visto en clase, o
 - El **esquema avanzado** subido a moodle

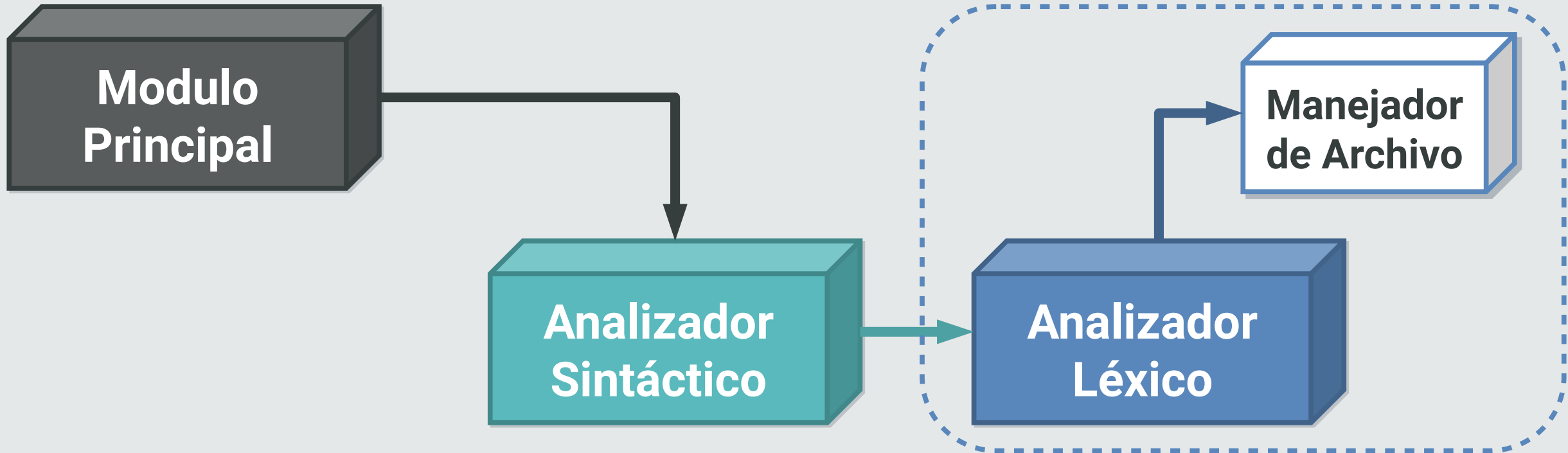
Estructura y Funcionalidad del Software a Desarrollar

Esquema General



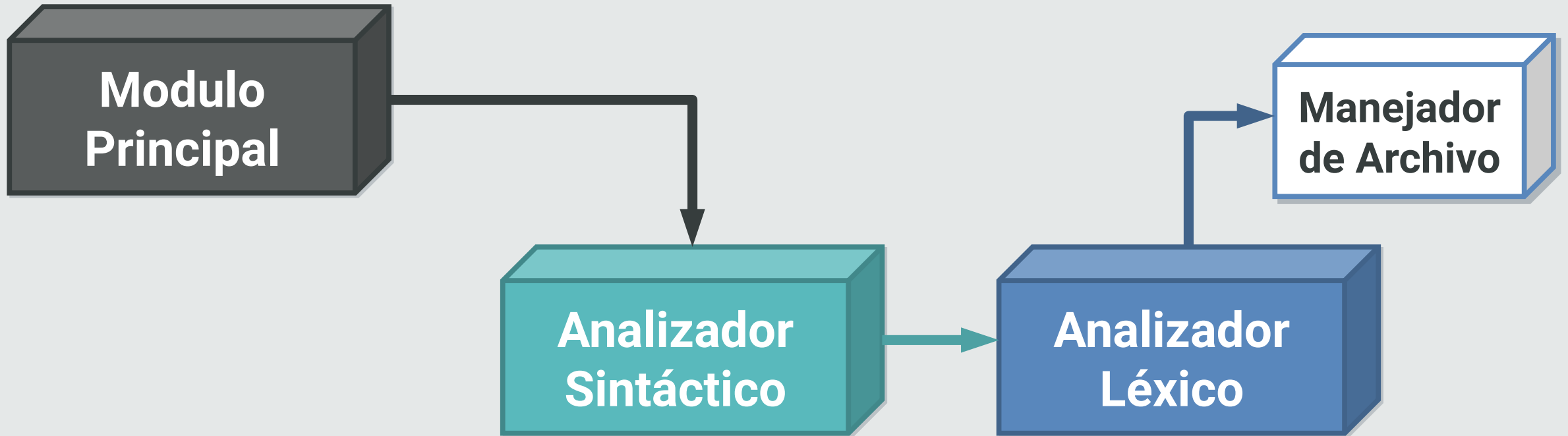
La base del proyecto
consistirá en estos cuatro
módulos

Esquema General



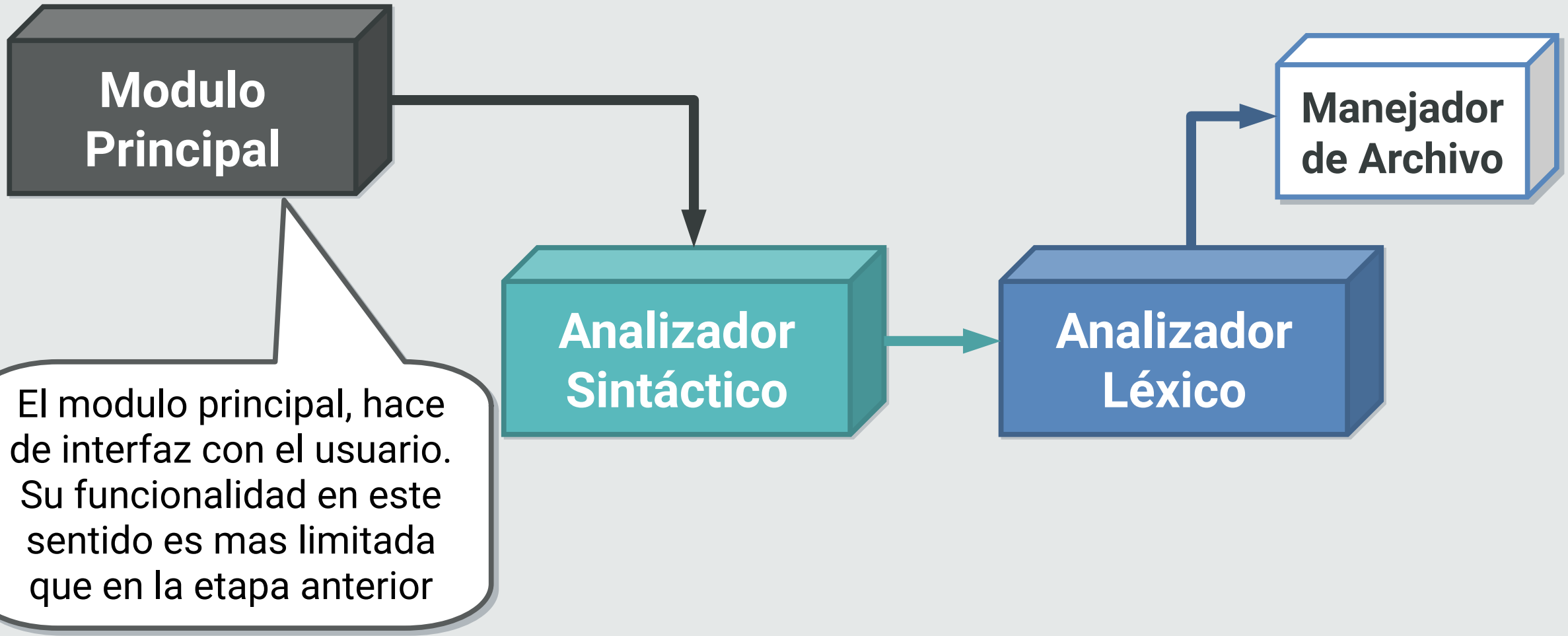
El Léxico y deberían ser los desarrollados en la etapa anterior (a lo sumo con sus bugs corregidos)

Esquema General



El sintáctico deberá implementarse según lo que vimos en la sección anterior

Esquema General



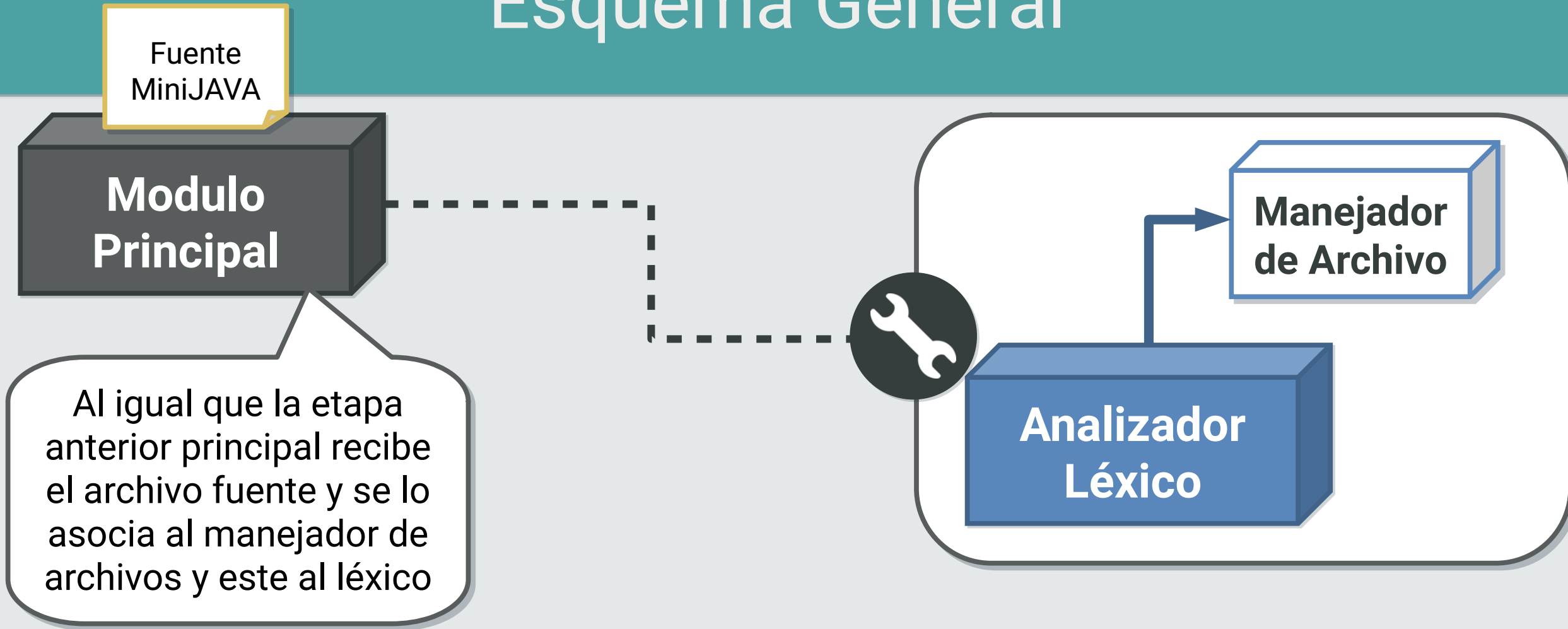
Esquema General

Fuente
MiniJAVA

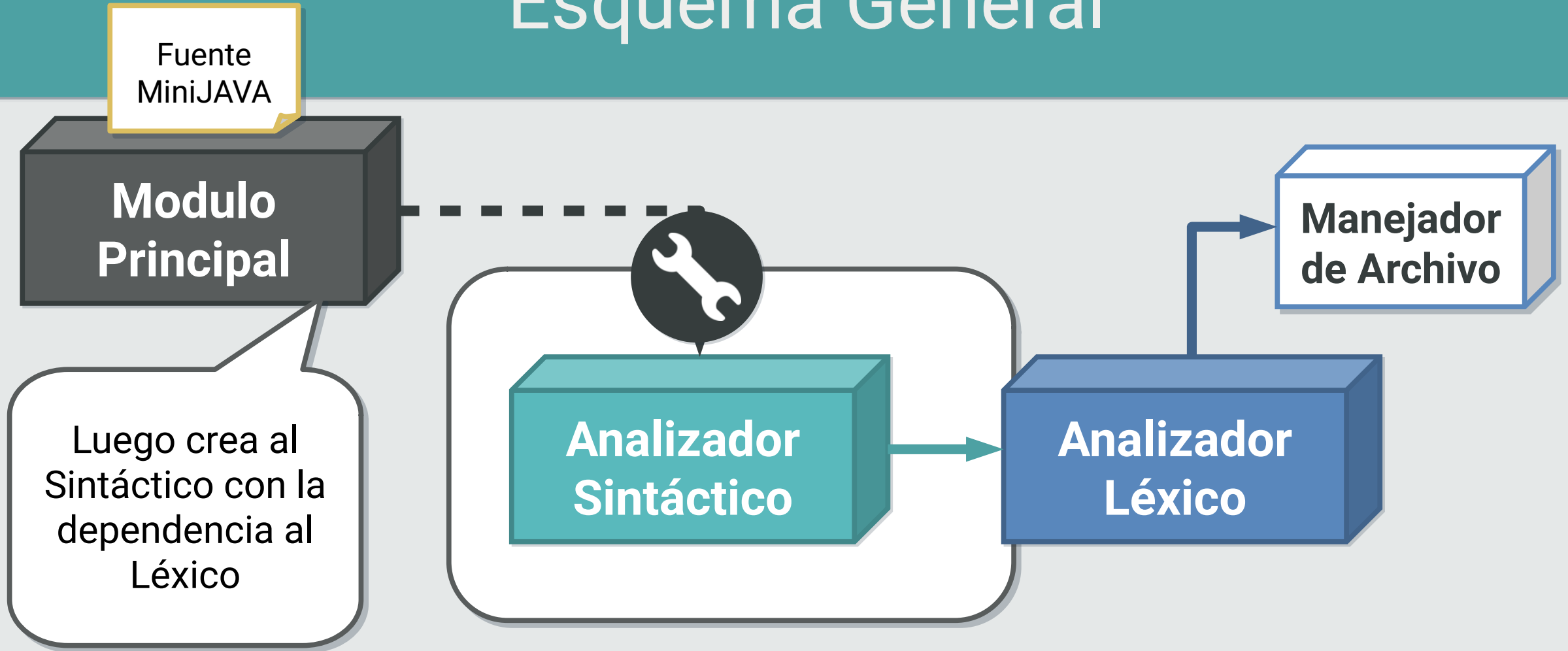
**Modulo
Principal**

Al igual que la etapa anterior principal recibe el archivo fuente y se lo asocia al manejador de archivos y este al léxico

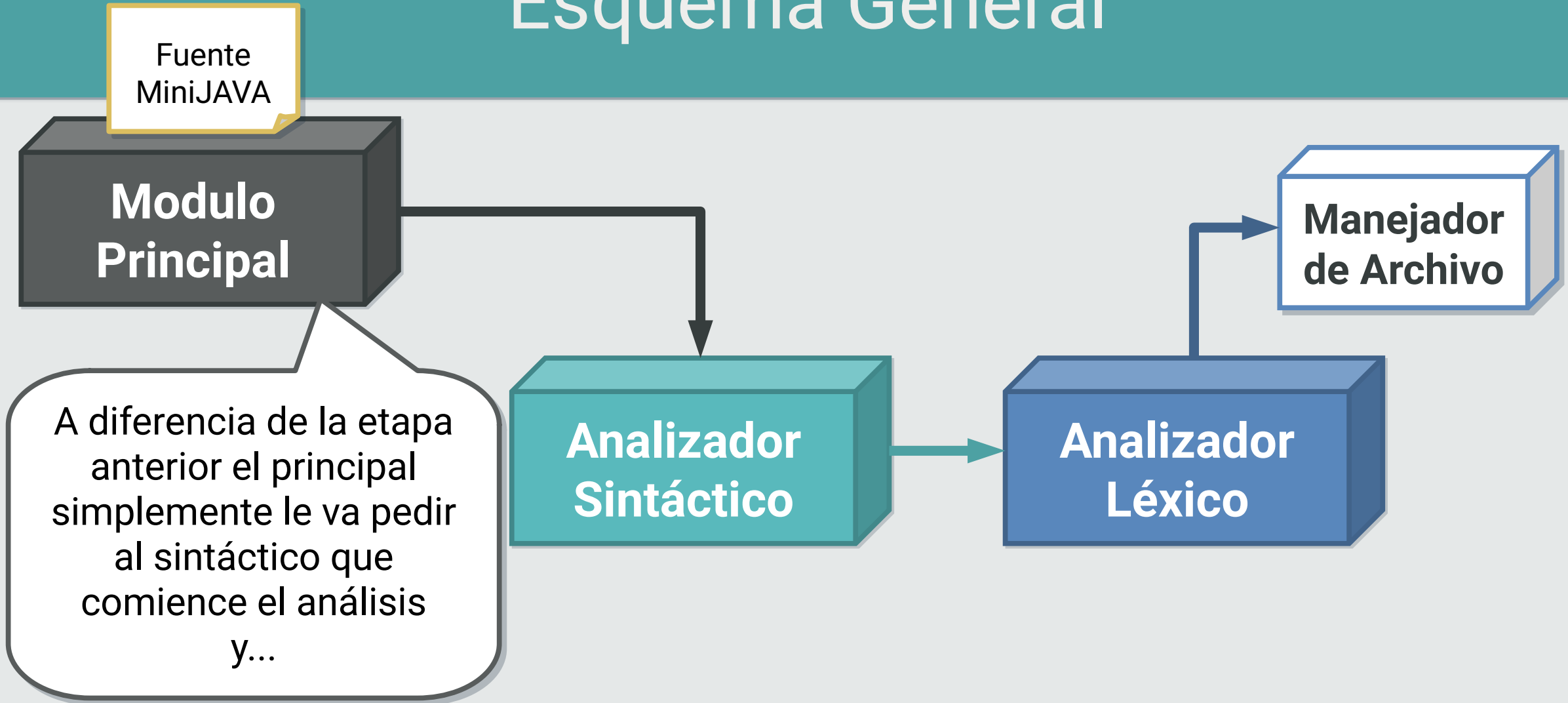
Esquema General

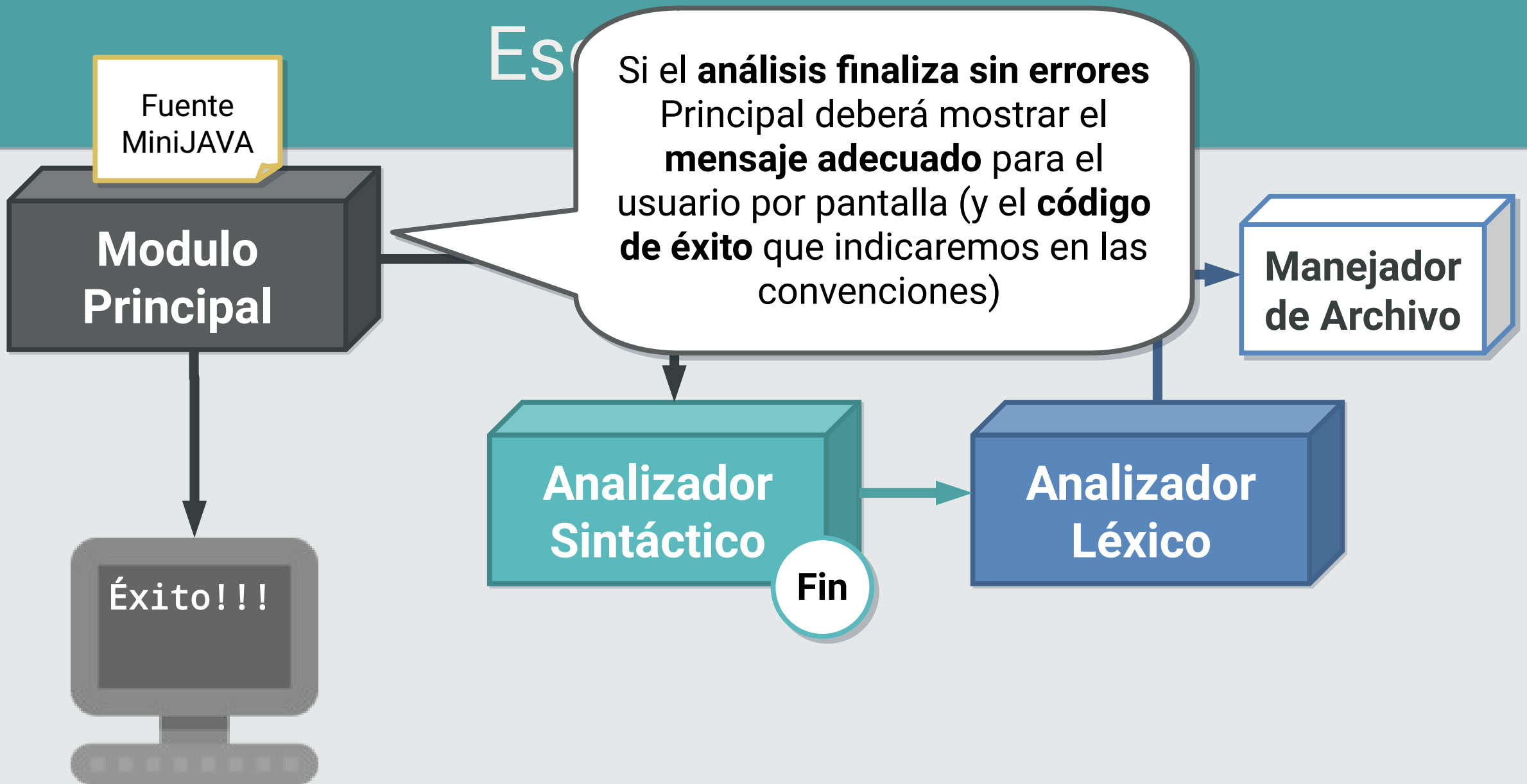


Esquema General



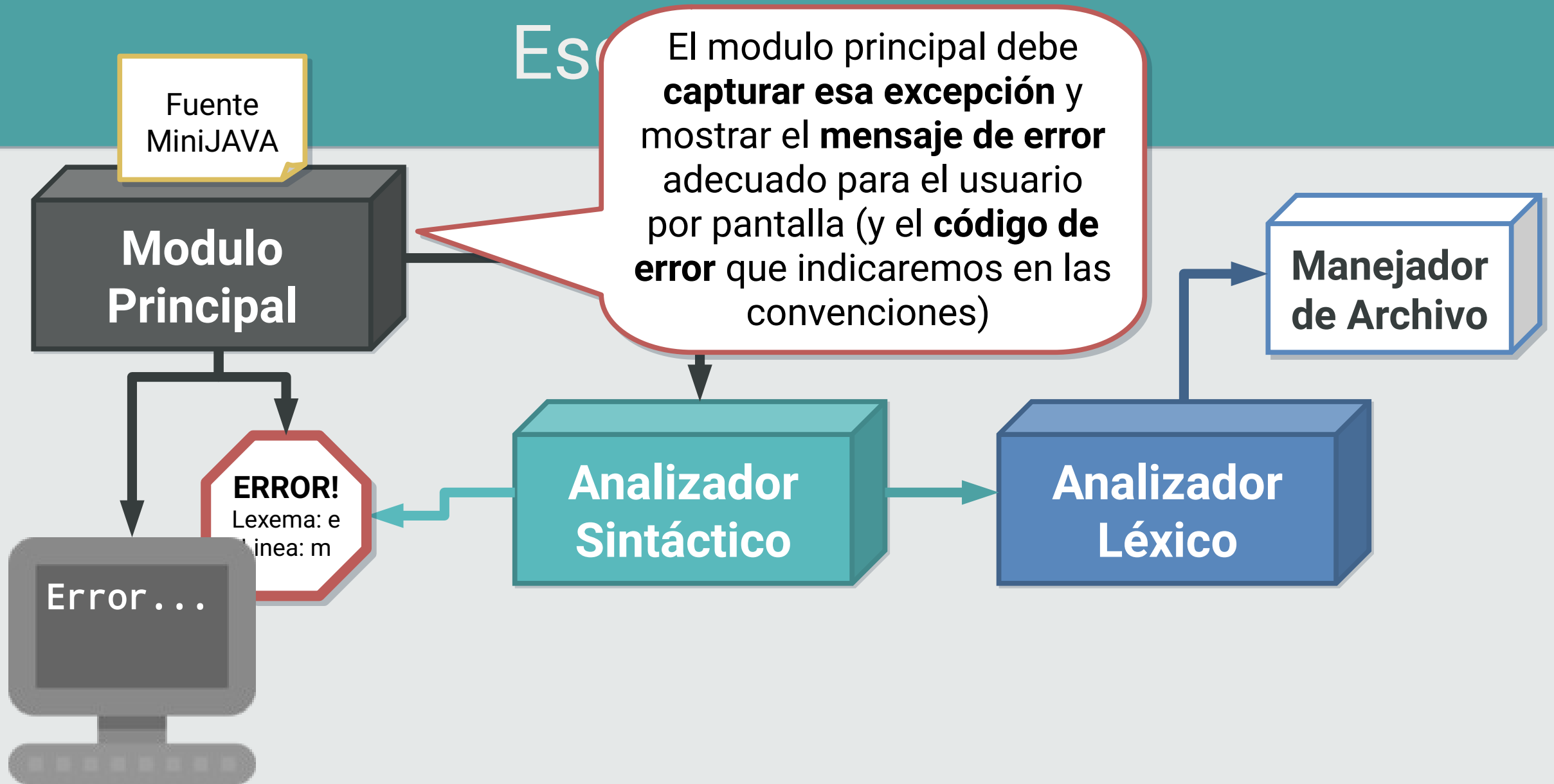
Esquema General





Esquema





Convenciones y Consideraciones

Convenciones y Consideraciones

Fecha Limite de Entrega: 23/09/2026 a las 16hs

Forma de Entrega: En la actividad asociada a la entrega etapa 2 en Moodle, subiendo un zip con todo lo requerido

Convenciones y Consideraciones

Reportes de Errores

- Al detectar un error deberá reportarse y finalizar la ejecución
- Deberá reportarse un mensaje adecuado a la naturaleza del error indicando el lexema del token con el que se produjo el error y en que numero de línea se encuentra.
- Al final del reporte deberá mostrarte un código de error con la siguiente estructura **[Error:Lexema|NroLinea]** donde Lexema y Nro de Linea son los reportados en el mensaje arriba mencionado.
- Ejemplo:



Fuente MiniJava

```
class {  
  
}
```

El Reporte de
Error debería ser

```
Error Sintactico en linea 1: se  
esperaba un id de clase se encontro "{"  
[Error:{|1]
```

Convenciones y Consideraciones

Reportes de Errores

- Al detectar un error deberá reportarse y finalizar la ejecución
- Deberá reportarse un mensaje adecuado a la naturaleza del error indicando el lexema del token con el que se produjo el error y en que numero de línea se encuentra.
- Al final del reporte deberá mostrarte un código de error con la siguiente estructura **[Error:Lexema|NroLinea]** donde Lexema y Nro de Linea son los reportados en el mensaje arriba mencionado.
- Ejemplo:



Fuente MiniJava

```
class {  
  
}
```

El Reporte de
Error debería ser

```
Error Sintactico en linea 1  
esperaba un id de clase S  
[Error:{|1]
```

En caso de ser
necesario el lexema del
token EOF es \$

Convenciones y Consideraciones

Mensaje de Análisis Exitoso

- Cuando no se detectan errores en el análisis el Modulo Principal debería mostrar un mensaje adecuado y luego el código de éxito: **[SinErrores]**
- Por ejemplo

Fuente MiniJava

```
Class C1{  
  
}
```

El mensaje
debería ser

```
Compilacion Exitosa  
[SinErrores]
```



Convenciones y Consideraciones

Invocación del Compilador

- Al ser compilado el software entregado poder invocarse de consola recibiendo como parámetro el archivo fuente MiniJava (debe aceptar cualquier tipo de extensión!)
- Por ejemplo: si al compilarlo lo llamamos Compilador y el fuente MiniJava se llama programa1.java, debería poder invocarse de la siguiente manera



```
java -jar Compilador.jar programa1.java
```

Convenciones y Consideraciones



No cumplir con estas convenciones llevará a incurrir en fallos y/o a la desaprobación de la entrega/reentrega

Convenciones y Consideraciones



No cumplir con estas convenciones llevará a incurrir en fallos y/o a la desaprobación de la entrega/reentrega

Las convenciones son esenciales para que **nosotros podamos corregir** bien la entrega...
por eso somos tan estrictos!

Si no la podemos corregir
va a estar **desaprobada!!!**

Herramientas de Test para la Evaluación

- En la tarea también les adjuntamos **herramientas de testing** de JUnit
- Estas herramientas las utilizamos en la cátedra como parte de la **evaluación** de los proyectos
- En el adjunto van a encontrar una **guía** de como agregarlas al proyecto (está armada respecto a **IntelliJ IDEA**)

Aclaración Importante: Las herramientas **no incluyen** los **casos de prueba** necesarios para **verificar adecuadamente** el compilador. El desarrollo de los casos es **responsabilidad del estudiante**. *Ademas, estas no son las únicas herramientas que utiliza la cátedra para la evaluación*

Informe

Informe

- El informe deberá indicar **como compilar el código fuente** entregado y como correrlo una vez compilado.
- Deberá reflejarse el **proceso de transformación de la gramática** de MiniJava para la implementación del Sintáctico
 - Debe incluirse la **versión final de la gramática** modificada que se utilizo
- Incluir los **tipos de errores sintacticos** que el analizador puede de detectar.
- Deberá incluirse la **documentación** usual, propia de un **proyecto de software** (decisiones de diseño, clases utilizadas, etc).
- Debe indicarse que **logros** se intentan alcanzar en la Etapa

Logros Disponibles para la Etapa 2

Logros - Aclaración

- Los logros que se presentarán a continuación están enfocados en características de los módulos a desarrollar en esta etapa
- Aun así es posible que algunos de estos logros requieran extender también las funcionalidades de lo desarrollado en etapas anteriores

Logros Etapa 2

MiniLambdas E2

El Analizador Sintáctico permite expresiones Lambda, con una sintaxis similar a la utilizada en Java. A diferencia de Java, los parámetros de una Lambda no pueden declarar tipo explícitamente y el cuerpo de una Lambda debe estar formado por una única expresión. No se permiten cuerpos delimitados por llaves. No se permite la captura de variables locales, parámetros, ni el uso de **this** dentro de una Lambda. Se permiten Lambdas con cualquier cantidad de parámetros, incluyendo ninguno.



Logros Etapa 2

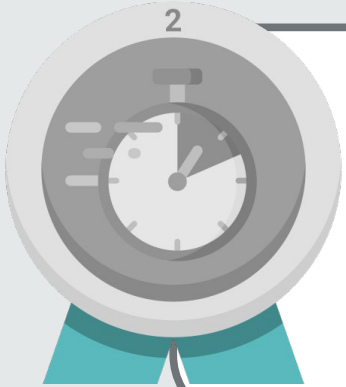


Variables locales Clásicas E2

El compilador permite declarar variables locales de la forma clásica de Java, es decir, sin usar **var** e indicando su tipo (por ej `int x;`). Además permite en una misma sentencia declarar varias variables e inicializarlas (por ejemplo `int x,y,z = 10;`)



Logros Etapa 2



Entrega Anticipada Sintáctica

(no valido para la reentrega, ni etapas subsiguiente)
La entrega debe realizarse 48hs antes de la fecha limite



Imbatibilidad Sintáctica

(no valido para la reentrega, ni etapas subsiguiente)
El software entregado pasa correctamente toda la batería de prueba utilizada por la cátedra

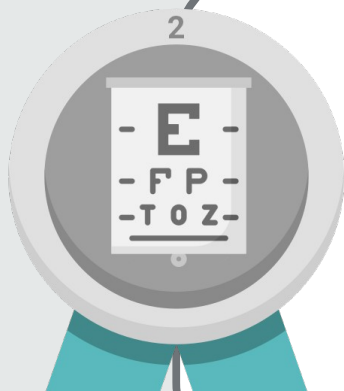
Logros Etapa 2



Well Done!

(no valido para la reentrega, ni etapas subsiguiente)

Gran trabajo! La entrega tiene solo algunos errores menores

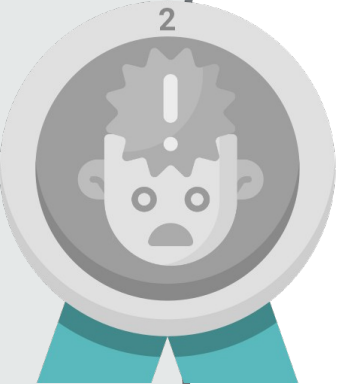


Visibilidad Mejorada E2

El analizador sintáctico permite indicar implícita y explícitamente la visibilidad de Métodos, Atributos y Constructores. De esta manera al declararse uno de los miembros se podrá obviar la visibilidad o indicarse explícitamente que es public o private.



Logros Etapa 2



Recuperación de Errores Modo Panico

El compilador no finaliza la ejecución ante el primer error sintáctico, sino que se recupera (Modo Pánico) y es capaz de reportar otros errores. Para ello, al encontrar un error, el analizador descarta la entrada hasta encontrar un token de sincronización que le permita reanudar el análisis. Se espera que se sincronice con el siguiente punto y coma, llave de cierre, o llave que abre segun el contexto.

Logros Etapa 2



Fors! E2

El compilador permite declarar sentencias de tipo **for** similar a como es en java. Se restringe a una sola sentencia/expresión por separador del for (“;” en el caso del standard y “:” en el for de iteradores)

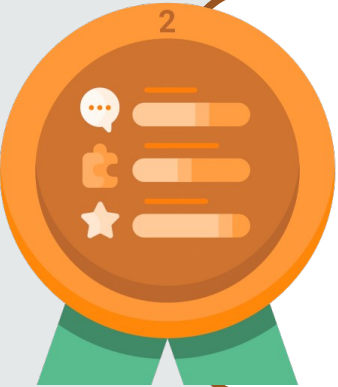


Genericidad Avanzada E2

El Analizador Sintáctico permite Tipos Genéricos Anidados y la notación Diamante, de forma similar a Java. Aun así, una Clase o Interfaz continúa limitada a un único Parámetro de Tipo. La notación Diamante solo puede utilizarse al instanciar una Clase Genérica.



Logros Etapa 2



Atributos Inicializados

El compilador permite realizar inicializaciones de los atributos cuando son declarados, al igual que en Java



Inicializacion de Arreglos E2

El Analizador Sintáctico permite la inicialización de Arreglos al momento de su construcción como en Java. Solo se permite como parte de una expresión de creación utilizando **new**.

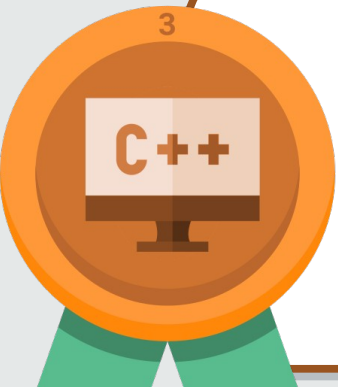


Logros Etapa 2



Operador Ternario E2

El compilador permite el uso del operador condicional ternario dentro de las expresiones como en Java (condición ? expr_verdadera : expr_falsa), permitiendo construcciones como, por ejemplo, `esMayor = (edad >= 18) ? true : false;`.



Operadores Posifijos E2

El Analizador Sintáctico permite usar los operadores unarios postfijos de incremento (`++`) y decremento (`--`).



Logros de Etapas Anteriores

- Recordar que es posible también intentar alcanzar **logros** de la **etapa anterior**
- Aun así chequear que el logro de sea alcanzable en una etapa posterior ya que existen algunos que no lo permiten (como por ejemplo entrega anticipada o imbatibilidad)